

Modül 14: Stream'ler

Bir stream (akış) bir boru hattıdır, koleksiyon değil. Hiç veri tutmaz ve bir kez tüketilir. Stream'lerle ilgili şaşırtıcı olan neredeyse her şey bu iki olgudan çıkar.

Modül 13 parçaları kurdu. Bir stream boru hattındaki her lambda o modüldeki arayüzlerden biri, dolayısıyla karşılığını almaya burada başlıyorlar.

Neler Öğreneceksiniz

- Boru hattı anatomisi ve iddia edilerek değil gösterilerek tembellik
- Elemanların neden aşama aşama değil teker teker aktığı
- Collector'lar, API'yi haklı çıkaran `groupingBy` ile birlikte
- Özel ara işlemlerin ait olduğu boşluğu dolduran gatherer'lar
- Paralel stream'ler, ne zaman yardım ettikleri ve onları tehlikeli kılan tuzak

Başka Bir Dilden Geliyorsanız

	Python	Java
Dönüştürme	comprehension, <code>map</code>	<code>.map(...)</code>
Filtreleme	comprehension <code>if</code> , <code>filter</code>	<code>.filter(...)</code>
Tembel	generator'lar	stream'ler, her zaman
Tekrar kullanım	liste iki kez yinelenebilir	bir stream yinelenemez
Gruplama	<code>itertools.groupby</code> sıralama ister	<code>Collectors.groupingBy</code> istemez
Paralel	<code>multiprocessing</code>	<code>.parallel()</code> , ve önce düşünün

Isıran satır tekrar kullanım. Python liste comprehension'ı size bir liste verir. Java stream'i, bir kez kullanıldığında buharlaşan bir boru hattı verir.

Ders

Anatomi

```
words.stream()           // SOURCE
  .filter(w -> w.length() > 4) // INTERMEDIATE, lazy
  .map(String::toUpperCase) // INTERMEDIATE, lazy
  .toList();              // TERMINAL, runs everything
```

Ara işlemler bir stream döndürür ve hiçbir şey yapmaz. Boru hattını gerçekte çalıştıran terminal işlemdir.

Tembellik, gösterilerek

`StreamBasics.java` dosyasından, içinde `peek` olan bir boru hattı kurmak hiçbir şey yazdırmaz:

```
pipeline built. notice nothing was peeked.  
now adding a terminal operation:  
peeked at banana  
...
```

Tembellik **kısa devreyi** mümkün kılan şeydir:

```
examined 1  
examined 2  
examined 3  
found Optional[3]
```

Eleman 4 ve 5'e hiç dokunulmadı. Sonsuz bir stream'in çalışmasının sebebi de bu:

```
Stream.iterate(1, n -> n + 1).map(n -> n * n).limit(5).toList()
```

Aşama aşama değil eleman eleman

Yaygın bir yanlış okuma, `filter` metodunun her şey üzerinde çalışıp sonra `map` metodunun her şey üzerinde çalıştığıdır. Öyle değil. Her eleman, bir sonraki başlamadan **boru hattının tamamından** geçer:

```
filter sees a  
map sees a  
forEach sees a  
filter sees b  
...
```

Collector'lar

`CollectingResults.java` dosyasından. Tek başına `groupingBy` API'yi öğrenmeyi haklı çıkarır:

```
STAFF.stream().collect(Collectors.groupingBy(Employee::department))
```

Aşağı akış collector'ı ikinci argümandır, ve API'nin laf kalabalığı olmaktan çıkıp değerli olmaya başladığı yer orasıdır:

```
groupingBy(Employee::department, Collectors.counting())  
groupingBy(Employee::department, Collectors.averagingInt(Employee::salary))  
groupingBy(Employee::department, TreeMap::new, Collectors.counting()) // ordered
```

Harita fabrikası olmadan sırası tanımsız olan bir `HashMap` alırsınız. Modül 12 burada da geçerli.

`partitioningBy` tam olarak iki gruba böler ve bir taraf boş olsa bile her zaman her iki anahtarı da taşır.

`toMap` tekrarlayan anahtarda ezmek yerine fırlatır:

```
duplicate key threw: Duplicate key engineering ...
```

Bu genellikle istediğiniz şeydir. Aksini söylemek için bir birleştirme fonksiyonu verin.

Stream üzerindeki `.toList()` değiştirilemez bir liste döndürür ve modern biçimdir.

`Collectors.toList()` tipi belirtilmemiş, değiştirilebilir bir liste döndürür. Birincisini tercih edin.

Gatherer'lar

JDK 24'te kesinleşti, dolayısıyla burada bayrak gerekmiyor. Gerçek bir boşluğu dolduruyorlar: bir `Collector` olarak özel bir **terminal** işlem her zaman yazabiliyordunuz, ama özel bir **ara** işlem asla. Elemanlar arasında durum tutan her şey stream'den çıkmak demekti.

`UsingGatherers.java` dosyasından:

Gatherer	1,2,3,4,5 üzerinde sonuç
<code>windowFixed(2)</code>	<code>[[1, 2], [3, 4], [5]]</code>
<code>windowSliding(2)</code>	<code>[[1, 2], [2, 3], [3, 4]]</code>
<code>scan(() -> 0, Integer::sum)</code>	<code>[1, 3, 6, 10]</code>

`windowFixed` son kısa pencereyi atmak yerine korur, ki toplu iş için istediğiniz budur: hiçbir kayıt kaybolmaz. `scan`, her ara değeri yayan `reduce` işlemidir, ki yürüyen bir bakiyenin ihtiyacı olan şey budur.

`mapConcurrent(n, fn)` sınırlı eşzamanlılıkla eşler **ve sırayı korur**, her birini bir sanal iş parçacığında çalıştırarak. Bu, IO biçimli işler için `.parallel()` seçeneğinden daha iyi bir varsayılandır, ve sonraki bölüm nedenini açıklıyor.

Ne zaman birine uzanmalı: elemanlar arasında durum, ya da tükettiğinizden farklı sayıda eleman yaymak. Bir girer bir çıkar, hafızasız ise o sadece `map`.

Paralel stream'ler

`ParallelStreams.java` dosyasından. `.parallel()` eklemek tek bir kelime, ve neredeyse hiçbir zaman doğru karar değil.

Aynı makinede üç kez çalıştırılan tuzak:

```
the list now holds: 23839
the list now holds: 22694
the list now holds: 84097
```

100.000 eleman paralel olarak bir `ArrayList` içine eklendi. Her çalıştırma farklı bir sayı kaybetti, ve dördüncü bir çalıştırma bunun yerine fırlatabilir, bir iş parçacığı diziyi yeniden boyutlandırırken bir diğeri

yazmanın ortasındaysa.

Tutarlı biçimde yanlış değil, ki bu tutarlı biçimde yanlış olmaktan daha kötü. Küçük girdide ya da yüksüz bir makinede genellikle çalışır. Bunun üretime ulaşma biçimi budur.

Çözüm kilit değil, `collect` :

```
IntStream.range(0, 100_000).parallel().boxed().toList(); // 100000, always
```

Her iş parçacığı kendi kabına biriktirir ve sonuçlar sonunda birleştirilir, dolayısıyla yazılırken hiçbir şey paylaşılmaz. `add` metodunu senkronize etmek de doğru olurdu ve ardışık sürümden *daha yavaş* olurdu, çünkü her iş parçacığı tek bir kilitle sıraya girerdi.

Paralelde `forEach` sırayı korumaz; `forEachOrdered` korur.

<code>.parallel()</code> düşünün	Kaçının
büyük N, eleman başına gerçekten pahalı iş	küçük N: fork/join maliyeti baskın gelir
kaynak ucuza bölünüyor: dizi, <code>ArrayList</code> , <code>IntStream.range</code>	<code>LinkedList</code> ya da yineleyici kaynaklar
paylaşılan durum yok, sıra gereksinimi yok	IO yapan her şey

IO durumu vurgulanmayı hak ediyor. Ortak havuz CPU çekirdeklerine göre boyutlandırılmıştır ve onu bloke etmek JVM'deki diğer her paralel stream'i aç bırakır. Onun yerine `Gatherers.mapConcurrent` ya da Modül 19'un sanal iş parçacıklarını kullanın.

Dürüst varsayılan: bir profilleyici bu boru hattını darboğaz olarak isimlendirene kadar `.parallel()` yazmayın.

Tek kullanım

`StreamsAreSingleUse.java` :

```
IllegalStateException: stream has already been operated upon or closed
```

Bir stream tekrar oynatacak kendi elemanlarını tutmaz. Stream'i değil **kaynağı** saklayın, ve bir koleksiyon size istediğiniz kadar stream verir.

Çalıştırın

```
java modules/14-streams/examples/StreamBasics.java
java modules/14-streams/examples/CollectingResults.java
java modules/14-streams/examples/UsingGatherers.java
java modules/14-streams/examples/ParallelStreams.java

# Bilerek başarısız olur.
java modules/14-streams/examples/StreamsAreSingleUse.java

./scripts/verify-examples.sh modules/14-streams
```

Sık Yapılan Hatalar

Bir stream'i alanda ya da değişkende saklayıp iki kez kullanmak. Kaynağı saklayın.

`forEach` içinden paylaşılan bir koleksiyonu değiştirmek. `collect` kullanın. Paralelde bu bir veri yarışdır; ardışıkta bile toplanmış sürümden daha zor okunur.

Bir şeyi hızlandırmak için `.parallel()` eklemek. Önce ölçün. Sıklıkla daha yavaş ve ara sıra yanlış.

Paralel stream içinde bloke olmak. Tüm JVM'in paylaştığı bir havuzdan iş parçacığı aldınız.

Gerçek yan etkiler için `peek` kullanmak. O bir hata ayıklama penceresidir. Uygulamaların, sonuç gerekmediğinde onu atlamasına izin verilmiştir.

`toMap` metodunun tekrarlarda fırlattığını unutmak. Anahtarlar tekrarlanabildiğinde bir birleştirme fonksiyonu verin.

Yerine geçtiği döngüden daha uzun bir boru hattı. Stream'ler *ne* olduğunu ifade etmek içindir, döngülerin yasak olduğu bir kural değil. Döngü daha netse onu yazın.

Özet Çıkarımlar

- **Bir stream tembel bir boru hattıdır, bir kez tüketilir.** Stream'i değil kaynağı saklayın.
- **Terminal işleme kadar hiçbir şey çalışmaz,** ki kısa devreyi ve sonsuz kaynakları mümkün kılan budur.
- **Elemanlar boru hattının tamamından teker teker akar,** aşama aşama değil.
- **Aşağı akış collector'ı ile `groupingBy`,** API'deki en kullanışlı şeydir.
- **Gatherer'lar size özel ara işlemler verir,** pencereleme ve yürüyen toplamlar için.
- **Paylaşılan değiştirilebilir durumla `.parallel()` veriyi öngörülemez biçimde kaybeder.** `collect` kullanın, ve paralelleştirmeden önce ölçün.

Ödev

[homework/README.md](#)

Bir log'u stream boru hattıyla analiz edin, bir gatherer kullanın ve paralel veri yarışını yeniden üretin.
Referans çözüm [solutions/14-streams/](#) altında.